

School of Computer Science and Engineering

Fall Semester 2026-27

BCSE331P – Exploratory Data Analysis

Assessment 2

NAME: ARJUN MISHRA

REG NO: 23BDS0334

#Load the data set

CODE:

assessment_2.R* × data ×

Source on Save

```
1  #Arjun Mishra
2  #23BDS0334
3  data <- read.csv("assessment.csv")
4
5  head(data)
6  str(data)
7  summary(data)
```

OUTPUT:

assessment_2.R* × data ×

Filter

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	Water	Cobble
1	1	0	786	190	10	0.1	0	3	0	0	0
2	2	0	794	182	10	0.0	0	3	0	0	0
3	3	0	798	178	20	0.0	0	1	0	0	0
4	4	0	800	176	70	0.3	0	1	0	1	0
5	5	0	804	172	40	0.3	0	1	0	1	0
6	6	0	808	168	60	0.3	0	1	0	1	0
7	7	0	814	162	40	0.3	0	1	0	1	0
8	8	0	817	159	0	0.9	1	1	1	1	0
9	9	0	821	155	50	0.5	0	1	0	1	0
10	10	0	826	150	20	0.1	0	1	0	0	0
11	11	0	829	147	0	0.7	0	1	1	1	0
12	12	0	834	142	10	0.9	1	1	0	1	0
13	13	0	839	137	0	0.3	1	5	0	1	0
14	14	0	842	134	0	0.0	0	4	0	0	0
15	15	0	844	132	10	0.0	0	3	0	0	0
16	16	0	845	131	20	0.9	2	1	0	1	0
17	17	0	849	127	50	0.4	0	1	0	1	0
18	18	0	851	125	50	0.9	0	1	0	0	0
19	19	0	854	122	0	0.8	1	1	0	1	0

Showing 1 to 20 of 1,592 entries, 11 total columns

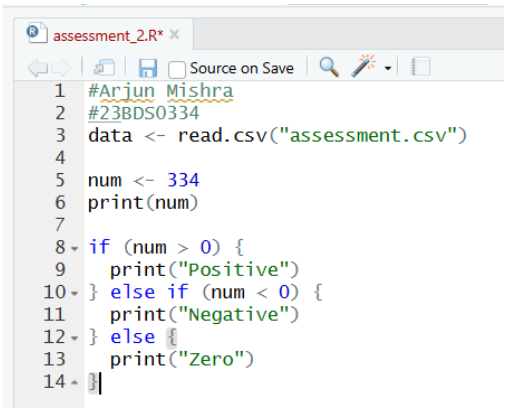
Assesment-1

Branching, Looping, and Functions in R Programming

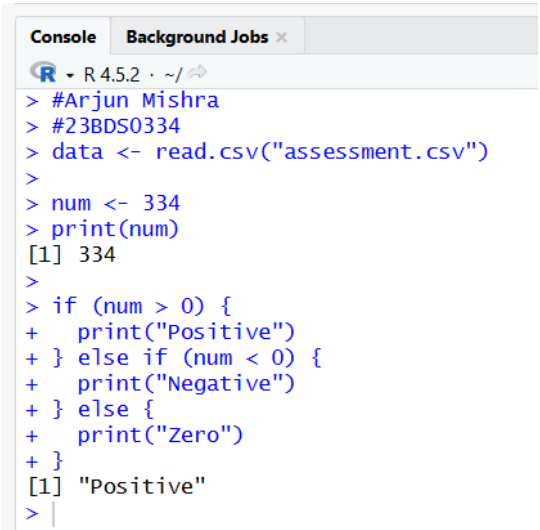
1. **Write an R program using an if-else statement to determine whether a given number is positive, negative, or zero.**

AIM: To write an R program using an if-else statement to determine whether a given number is positive, negative, or zero.

CODE:



OUTPUT:



RESULT: Thus, the given number 334 was successfully identified as a positive number using an if-else statement.

2. **Create a for loop that iterates through a vector of numbers and prints twice the value of each element.**

AIM:

To create a for loop that iterates through a vector of numbers and prints twice the value of each element.

CODE:

assessment_2.R*

Source on Save

```r
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

numbers <- c(334, 335, 336, 337, 338)

for (i in numbers) {
  print(i * 2)
}
```

OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> numbers <- c(334, 335, 336, 337, 338)
>
> for (i in numbers) {
+   print(i * 2)
+ }
[1] 668
[1] 670
[1] 672
[1] 674
[1] 676
>
```

RESULT:

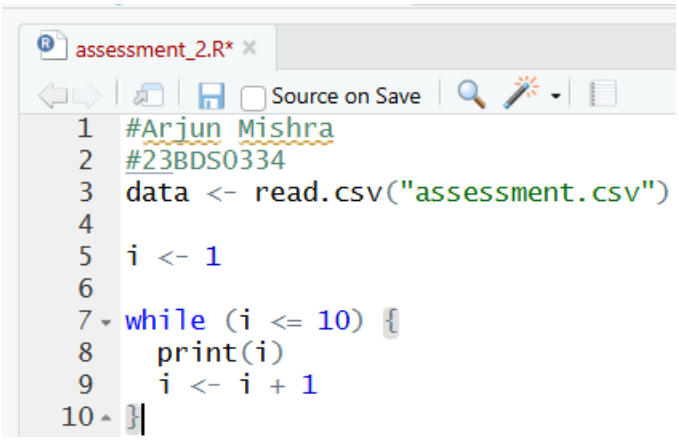
Thus, the for loop successfully calculated and displayed twice the value of each element.

3. **Write an R program using a while loop to print the numbers from 1 to 10. Explain the condition that terminates the loop.**

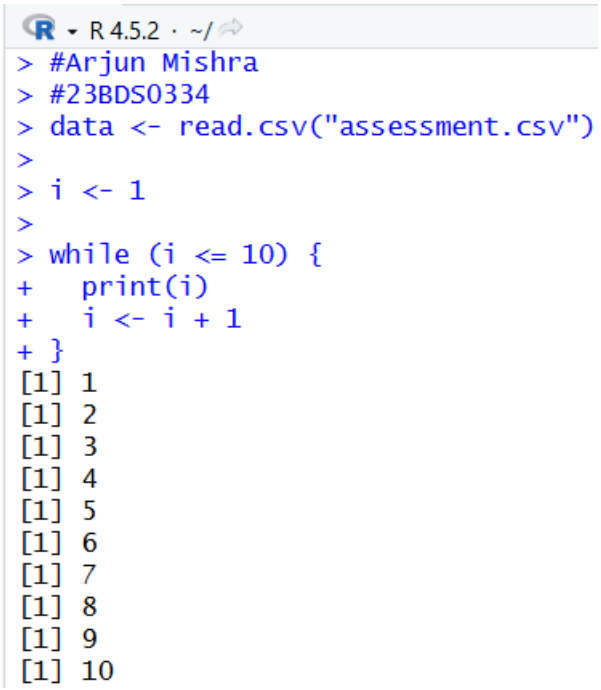
AIM:

To write an R program using a while loop to print numbers from 1 to 10.

CODE:



OUTPUT:



RESULT:

Thus, numbers from 1 to 10 were successfully displayed using a while loop. The loop terminates when i becomes greater than 10.

4. **Use a repeat loop to print the message "Learning R" five times. Include an appropriate condition and a break statement to terminate the loop.**

AIM:

To use a repeat loop to print "Learning R" five times and terminate it using a break statement.

CODE:

assessment_2.R*

Source on Save

```r
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

name <- "Arjun Mishra"
print(name)

i <- 1

repeat {
  print("Learning R")
  i <- i + 1

  if (i > 5) {
    break
  }
}
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> name <- "Arjun Mishra"
> print(name)
[1] "Arjun Mishra"
>
> i <- 1
>
> repeat {
+   print("Learning R")
+   i <- i + 1
+
+   if (i > 5) {
+     break
+   }
+ }
[1] "Learning R"
[1] "Learning R"
[1] "Learning R"
[1] "Learning R"
[1] "Learning R"
```

RESULT:

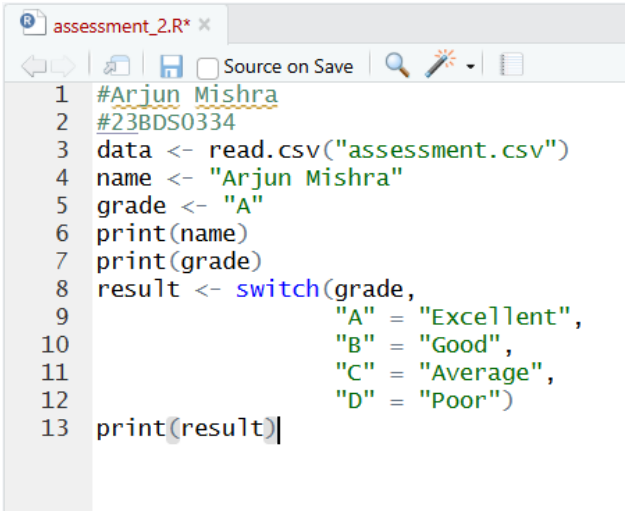
Thus, "Learning R" was successfully printed five times using a repeat loop and break statement.

5. **Write an R program using a switch statement that accepts a grade such as "A", "B", "C", or "D" and displays "Excellent", "Good", "Average", or "Poor", respectively.**

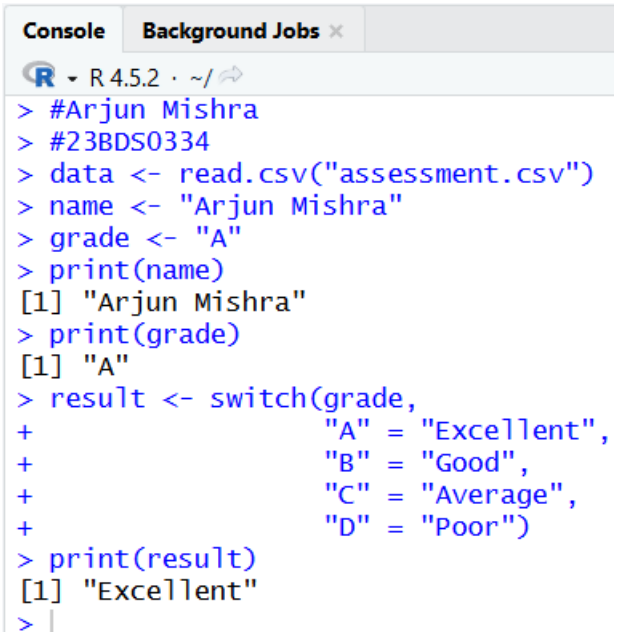
AIM:

To use a switch statement to display a message corresponding to a given grade.

CODE:



OUTPUT:



RESULT:

Thus, the switch statement successfully displayed **"Excellent"** for grade **"A"**.

6. Create a nested for loop to generate multiplication tables for the numbers 1 to 5, with each table containing multiples from 1 to 10.

AIM: To generate multiplication tables from 1 to 5 with multiples from 1 to 10 using nested for loops.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

for (i in 1:5) {
  for (j in 1:10) {
    print(paste(i, "x", j, "=", i * j))
  }
}
```

OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> for (i in 1:5) {
+   for (j in 1:10) {
+     print(paste(i, "x", j, "=", i * j))
+   }
+ }
[1] "1 x 1 = 1"
[1] "1 x 2 = 2"
[1] "1 x 3 = 3"
[1] "1 x 4 = 4"
[1] "1 x 5 = 5"
[1] "1 x 6 = 6"
[1] "1 x 7 = 7"
[1] "1 x 8 = 8"
[1] "1 x 9 = 9"
[1] "1 x 10 = 10"
[1] "2 x 1 = 2"
[1] "2 x 2 = 4"
[1] "2 x 3 = 6"
[1] "2 x 4 = 8"
[1] "2 x 5 = 10"
[1] "2 x 6 = 12"
[1] "2 x 7 = 14"
[1] "2 x 8 = 16"
[1] "2 x 9 = 18"
[1] "2 x 10 = 20"
[1] "3 x 1 = 3"
[1] "3 x 2 = 6"
[1] "3 x 3 = 9"
[1] "3 x 4 = 12"
[1] "3 x 5 = 15"
[1] "3 x 6 = 18"
[1] "3 x 7 = 21"
[1] "3 x 8 = 24"
[1] "3 x 9 = 27"
[1] "3 x 10 = 30"
[1] "4 x 1 = 4"
[1] "4 x 2 = 8"
[1] "4 x 3 = 12"
[1] "4 x 4 = 16"
[1] "4 x 5 = 20"
[1] "4 x 6 = 24"
[1] "4 x 7 = 28"
[1] "4 x 8 = 32"
[1] "4 x 9 = 36"
[1] "4 x 10 = 40"
[1] "5 x 1 = 5"
[1] "5 x 2 = 10"
[1] "5 x 3 = 15"
[1] "5 x 4 = 20"
[1] "5 x 5 = 25"
[1] "5 x 6 = 30"
[1] "5 x 7 = 35"
[1] "5 x 8 = 40"
[1] "5 x 9 = 45"
[1] "5 x 10 = 50"
>
```

RESULT:

Thus, multiplication tables from 1 to 5 were successfully generated using nested for loops.

7. **Write an R program using conditional statements to classify a person as a "Child", "Teenager", "Adult", or "Senior Citizen" based on their age.**

AIM:

To classify a person as Child, Teenager, Adult, or Senior Citizen using conditional statements.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

age <- data$Fromlow[1]
print(age)

if (age < 13) {
  print("Child")
} else if (age < 20) {
  print("Teenager")
} else if (age < 60) {
  print("Adult")
} else {
  print("Senior Citizen")
}
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> age <- data$Fromlow[1]
> print(age)
[1] 190
>
> if (age < 13) {
+   print("Child")
+ } else if (age < 20) {
+   print("Teenager")
+ } else if (age < 60) {
+   print("Adult")
+ } else {
+   print("Senior Citizen")
+ }
[1] "Senior Citizen"
>
```

RESULT:

Thus, the value obtained from the dataset was successfully classified using conditional statements.

8. Demonstrate the use of the break statement in a for loop by terminating the loop when a specified value is encountered.

AIM:

To demonstrate the use of the break statement by terminating a for loop when a specified value is encountered.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

numbers <- data$Slope[1:5]
stop_value <- numbers[4]

print(numbers)

for (i in numbers) {
  if (i == stop_value) {
    break
  }
  print(i)
}
```

OUTPUT:

15:2 (Top Level)

Console Background Jobs

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> numbers <- data$Slope[1:5]
> stop_value <- numbers[4]
>
> print(numbers)
[1] 10 10 20 70 40
>
> for (i in numbers) {
+   if (i == stop_value) {
+     break
+   }
+   print(i)
+ }
[1] 10
[1] 10
[1] 20
```

RESULT:

Thus, the break statement successfully terminated the loop at the specified value.

9. **Write an R program using the next statement to skip even numbers and print only the odd numbers between 1 and 10.**

AIM:

To use the next statement to skip even numbers and print only odd numbers between 1 and 10.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

for (i in 1:10) {
  if (i %% 2 == 0) {
    next
  }
  print(i)
}
```

OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> for (i in 1:10) {
+   if (i %% 2 == 0) {
+     next
+   }
+   print(i)
+ }
[1] 1
[1] 3
[1] 5
[1] 7
[1] 9
```

RESULT:

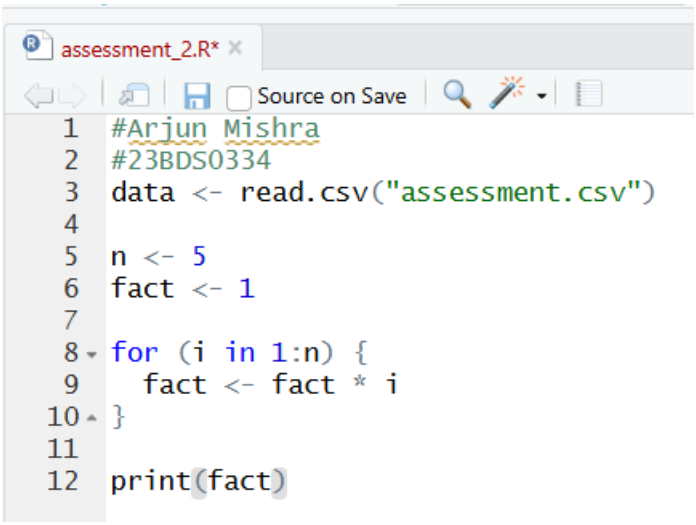
Thus, even numbers were successfully skipped and only odd numbers were displayed using next.

10. Create a for loop to calculate the factorial of a given non-negative integer.

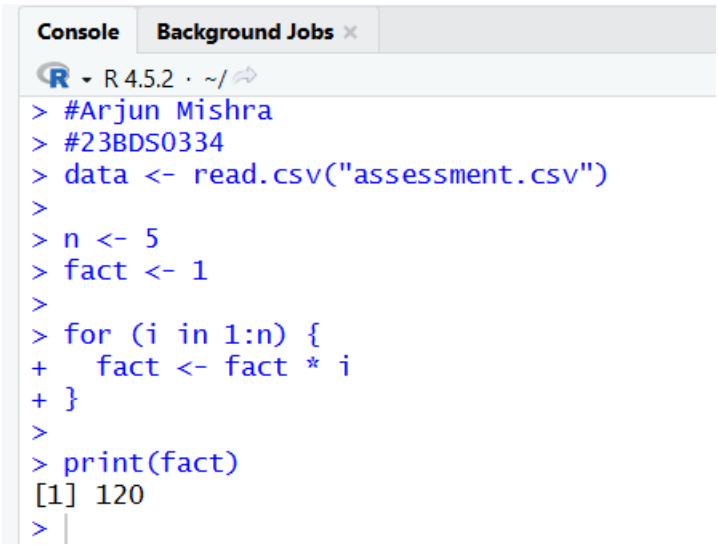
AIM:

To calculate the factorial of a given non-negative integer using a for loop.

CODE:



OUTPUT:



RESULT:

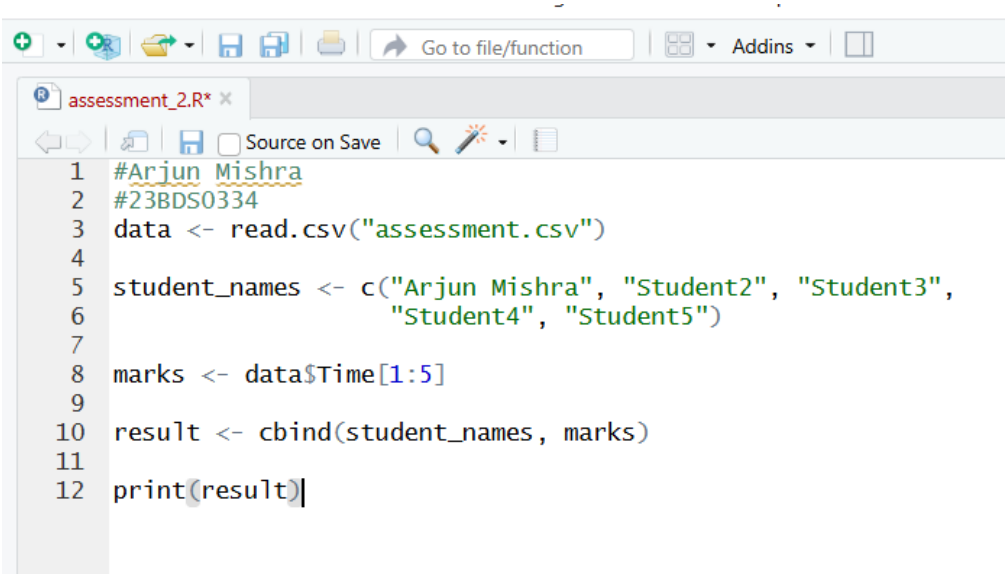
Thus, the factorial of 5 was successfully calculated as 120.

11. Create two vectors containing student names and marks. Use the cbind() function to combine them column-wise into a matrix.

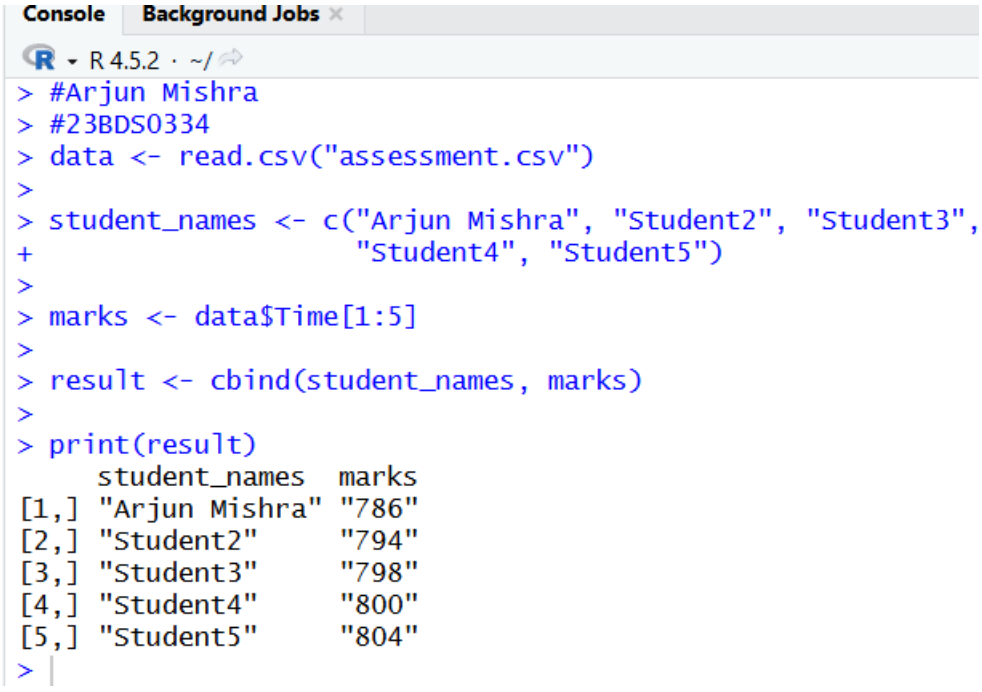
AIM:

To combine student names and marks column-wise using the cbind() function.

CODE:



OUTPUT:



RESULT:

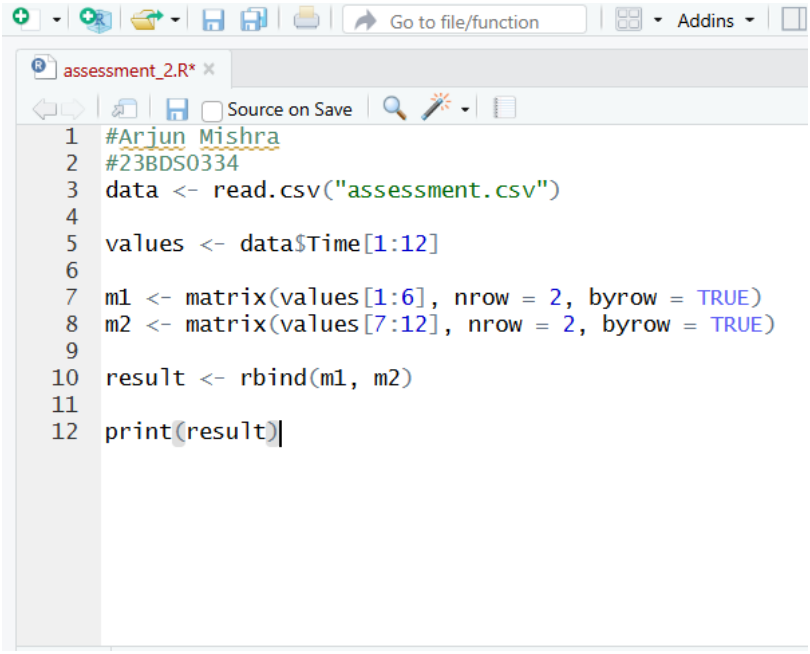
Thus, student names and dataset values were successfully combined column-wise using cbind().

12. Create two matrices with the same number of columns and combine them row-wise using the rbind() function. Display the resulting matrix.

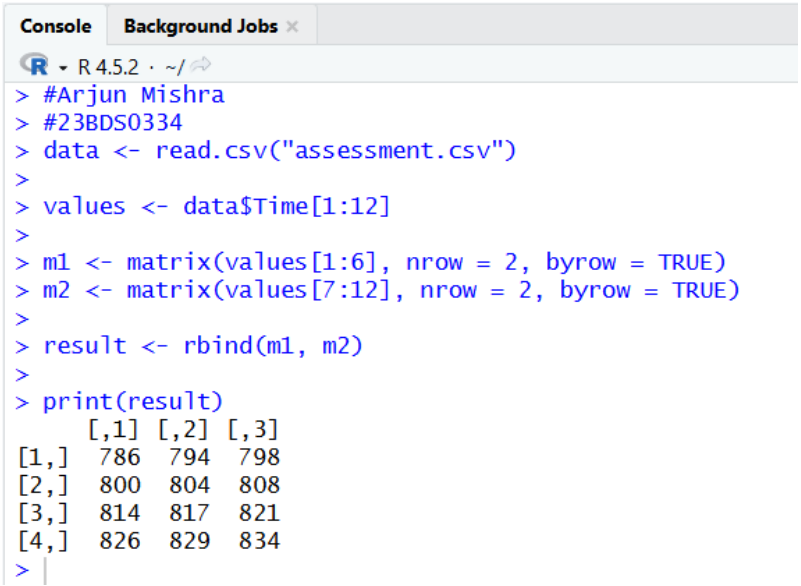
AIM:

To combine two matrices row-wise using the rbind() function.

CODE:



OUTPUT:



RESULT:

Thus, two matrices were successfully combined row-wise using rbind().

13. Create a numeric vector containing at least ten observations and use the summary() function to display its minimum, first quartile, median, mean, third quartile, and maximum values.

AIM:

To calculate the minimum, first quartile, median, mean, third quartile, and maximum of a numeric vector using summary().

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

numbers <- data$Time[1:10]

print(numbers)

result <- summary(numbers)

print(result)
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> numbers <- data$Time[1:10]
>
> print(numbers)
 [1] 786 794 798 800 804 808 814 817 821 826
>
> result <- summary(numbers)
>
> print(result)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  786.0   798.5   806.0   806.8   816.2   826.0
>
```

RESULT:

Thus, descriptive statistics of the selected dataset values were successfully obtained using summary().

14. Write an R program that demonstrates any five built-in mathematical functions, such as sqrt(), abs(), round(), ceiling(), floor(), log(), or exp().

AIM:

To demonstrate five built-in mathematical functions in R.

CODE:

assessment_2.R*

Source on Save

```
1  #Arjun Mishra
2  #23BDS0334
3  data <- read.csv("assessment.csv")
4
5  x <- 334
6
7  print(sqrt(x))
8  print(abs(-x))
9  print(round(334.56))
10 print(ceiling(334.25))
11 print(floor(334.75))
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> x <- 334
>
> print(sqrt(x))
[1] 18.27567
> print(abs(-x))
[1] 334
> print(round(334.56))
[1] 335
> print(ceiling(334.25))
[1] 335
> print(floor(334.75))
[1] 334
>
```

RESULT:

Thus, sqrt(), abs(), round(), ceiling(), and floor() were successfully demonstrated.

15. Define a user-defined function that accepts two numbers and returns their sum, difference, product, and quotient.

AIM:

To define a function that accepts two numbers and returns their sum, difference, product, and quotient.

CODE:

assessment_2.R*

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
a <- data$Time[1]
b <- data$Slope[1]

calculate <- function(a, b) {
  result <- c(
    Sum = a + b,
    Difference = a - b,
    Product = a * b,
    Quotient = a / b
  )
  return(result)
}
print(a)
print(b)
result <- calculate(a, b)
print(result)
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> a <- data$Time[1]
> b <- data$Slope[1]
>
> calculate <- function(a, b) {
+   result <- c(
+     Sum = a + b,
+     Difference = a - b,
+     Product = a * b,
+     Quotient = a / b
+   )
+   return(result)
+ }
> print(a)
[1] 786
> print(b)
[1] 10
> result <- calculate(a, b)
> print(result)
       Sum Difference    Product   Quotient
     796.0      776.0     7860.0       78.6
```

RESULT:

Thus, a user-defined function was successfully created to perform four arithmetic operations.

16. Create a user-defined function to determine whether a given number is prime. Test the function using at least three different numbers.

AIM:

To create a user-defined function to determine whether a number is prime and test it with three numbers.

CODE:

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
is_prime <- function(n) {
  if (n <= 1) {
    return(FALSE)
  }
  if (n == 2) {
    return(TRUE)
  }
  for (i in 2:floor(sqrt(n))) {
    if (n %% i == 0) {
      return(FALSE)
    }
  }
  return(TRUE)
}
print(is_prime(334))
print(is_prime(337))
print(is_prime(347))
```

OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv'
> is_prime <- function(n) {
+   if (n <= 1) {
+     return(FALSE)
+   }
+   if (n == 2) {
+     return(TRUE)
+   }
+   for (i in 2:floor(sqrt(n))) {
+     if (n %% i == 0) {
+       return(FALSE)
+     }
+   }
+   return(TRUE)
+ }
> print(is_prime(334))
[1] FALSE
> print(is_prime(337))
[1] TRUE
> print(is_prime(347))
[1] TRUE
>
```

RESULT:

Thus, the function successfully determined the prime-number status of the three given values.

17. Write an R program using the sub() function to replace the first occurrence of the word "R" with "R Programming" in a given character string.

AIM:

To replace the first occurrence of "R" with "R Programming" using sub().

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
text <- "Arjun Mishra is learning R"
print(text)
result <- sub("R", "R Programming", text)
print(result)
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> text <- "Arjun Mishra is learning R"
> print(text)
[1] "Arjun Mishra is learning R"
> result <- sub("R", "R Programming", text)
> print(result)
[1] "Arjun Mishra is learning R Programming"
>
```

RESULT:

Thus, the first occurrence of "R" was successfully replaced with "R Programming" using sub().

18. Write an R program using the gsub() function to replace all occurrences of a specified word or character in a given sentence.

AIM:

To replace all occurrences of a specified word in a sentence using gsub().

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
text <- "Arjun Mishra likes R and Arjun Mishra is learning R"
print(text)
result <- gsub("R", "R Programming", text)
print(result)
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> text <- "Arjun Mishra likes R and Arjun Mishra is learning R"
> print(text)
[1] "Arjun Mishra likes R and Arjun Mishra is learning R"
> result <- gsub("R", "R Programming", text)
> print(result)
[1] "Arjun Mishra likes R Programming and Arjun Mishra is learning R Programming"
>
```

RESULT:

Thus, all occurrences of "R" were successfully replaced with "R Programming" using gsub().

19. **Create a matrix containing numeric values and use the apply() function to calculate the sum and mean of each row and each column.**

AIM:

To calculate the sum and mean of each row and column of a matrix using apply().

CODE:

assessment_2.R*

Source on Save

```r
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
m <- as.matrix(data[1:5, c("Time", "Fromlow", "Slope")])
print(m)
row_sum <- apply(m, 1, sum)
column_sum <- apply(m, 2, sum)
row_mean <- apply(m, 1, mean)
column_mean <- apply(m, 2, mean)
print(row_sum)
print(column_sum)
print(row_mean)
print(column_mean)
```

OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> m <- as.matrix(data[1:5, c("Time", "Fromlow", "Slope")])
> print(m)
  Time Fromlow Slope
1  786     190    10
2  794     182    10
3  798     178    20
4  800     176    70
5  804     172    40
> row_sum <- apply(m, 1, sum)
> column_sum <- apply(m, 2, sum)
> row_mean <- apply(m, 1, mean)
> column_mean <- apply(m, 2, mean)
> print(row_sum)
   1    2    3    4    5
 986  986  996 1046 1016
> print(column_sum)
   Time Fromlow   Slope
   3982     898     150
> print(row_mean)
       1        2        3        4        5
328.6667 328.6667 332.0000 348.6667 338.6667
> print(column_mean)
   Time Fromlow   Slope
  796.4   179.6    30.0
```

RESULT:

Thus, the sum and mean of each row and column were successfully calculated using apply().

20. Create a list containing three numeric vectors and use lapply() and sapply() to calculate the mean of each vector. Compare the formats of the outputs returned by the two functions.

AIM:

To calculate the mean of three numeric vectors using lapply() and sapply() and compare their output formats.

CODE:

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
my_list <- list(
  data$Time[1:5],
  data$Slope[1:5],
  data$Fromlow[1:5]
)
print(my_list)
lapply_result <- lapply(my_list, mean)
sapply_result <- sapply(my_list, mean)
print(lapply_result)
print(sapply_result)
```

OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> my_list <- list(
+   data$Time[1:5],
+   data$Slope[1:5],
+   data$Fromlow[1:5]
+ )
> print(my_list)
[[1]]
[1] 786 794 798 800 804

[[2]]
[1] 10 10 20 70 40

[[3]]
[1] 190 182 178 176 172

> lapply_result <- lapply(my_list, mean)
> sapply_result <- sapply(my_list, mean)
> print(lapply_result)
[[1]]
[1] 796.4

[[2]]
[1] 30

[[3]]
[1] 179.6

> print(sapply_result)
[1] 796.4  30.0 179.6
>
```

RESULT:

Thus, the mean of each vector was successfully calculated using both lapply() and sapply(), demonstrating the difference in their returned formats.

ASSESSMENT 2

Data Import, Transform and Basic Statistical Analysis

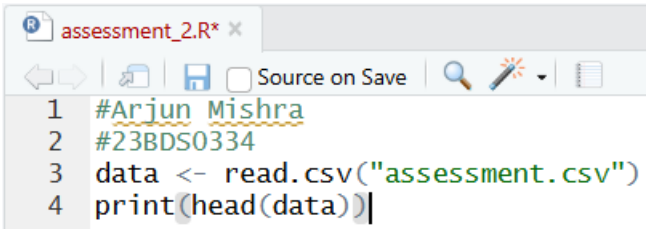
Write an R program to import the given dataset and perform preliminary data exploration.

1. **Import the given dataset (CSV/Excel) into R. (Dataset share through email)**

AIM:

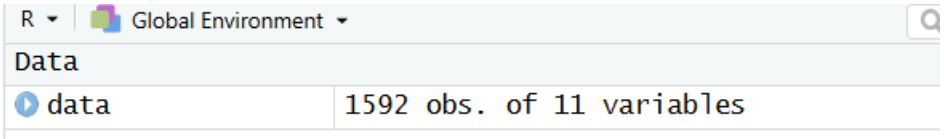
To import the given CSV dataset into R.

CODE:



OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> print(head(data))
  rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1        1      0  786     190    10 0.1       0     3    0     0      0
2        2      0  794     182    10 0.0       0     3    0     0      0
3        3      0  798     178    20 0.0       0     1    0     0      0
4        4      0  800     176    70 0.3       0     1    0     1      0
5        5      0  804     172    40 0.3       0     1    0     1      0
6        6      0  808     168    60 0.3       0     1    0     1      0
```



RESULT:

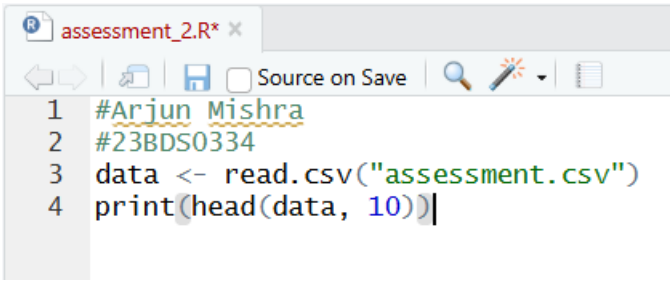
Thus, the CSV dataset was successfully imported into R.

2. Display the first 10 records of the dataset.

AIM:

To display the first 10 records of the dataset.

CODE:



OUTPUT:

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

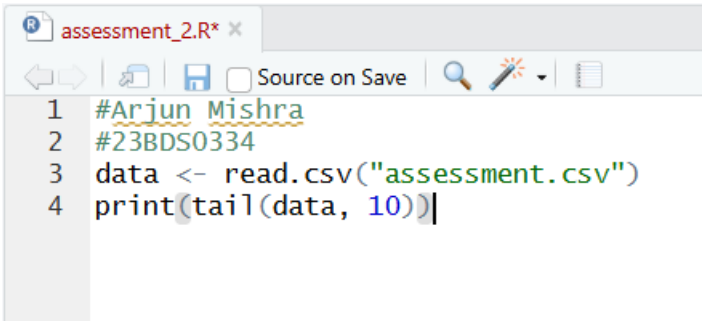
Thus, the first 10 records were successfully displayed.

3. **Display the last 10 records of the dataset.**

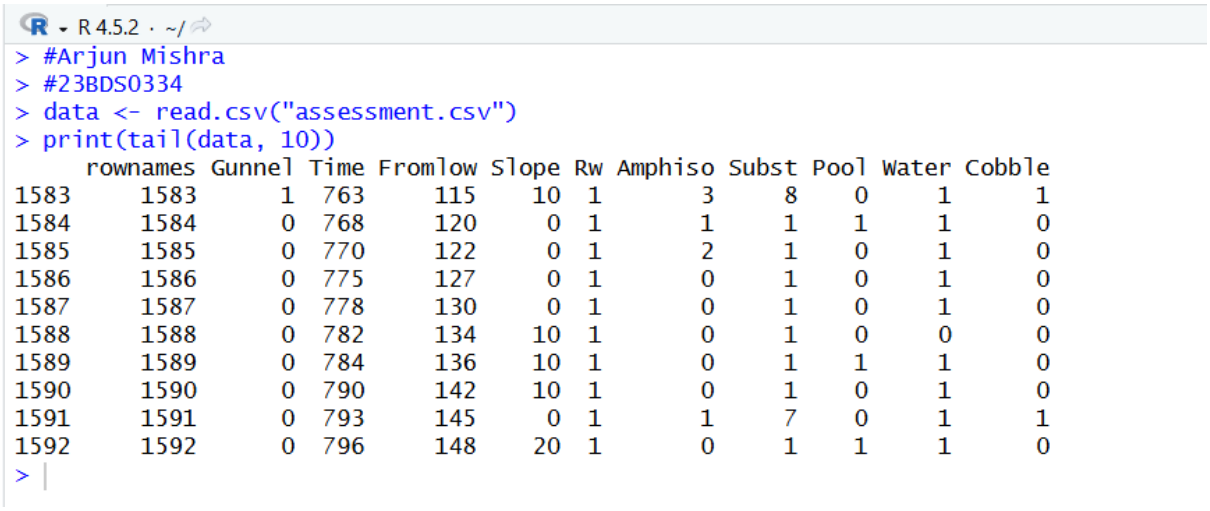
AIM:

To display the last 10 records of the dataset.

CODE:



OUTPUT:



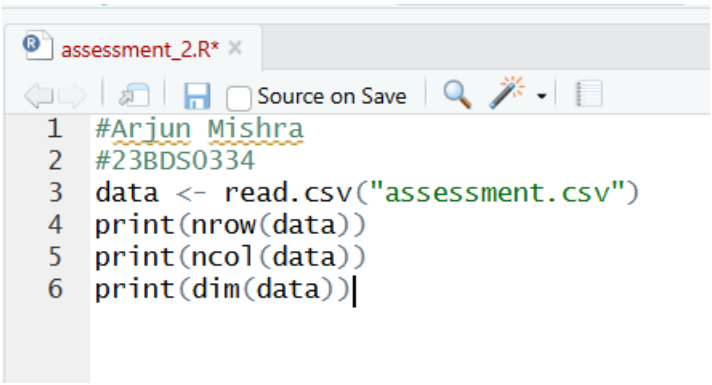
RESULT:

Thus, the last 10 records were successfully displayed.

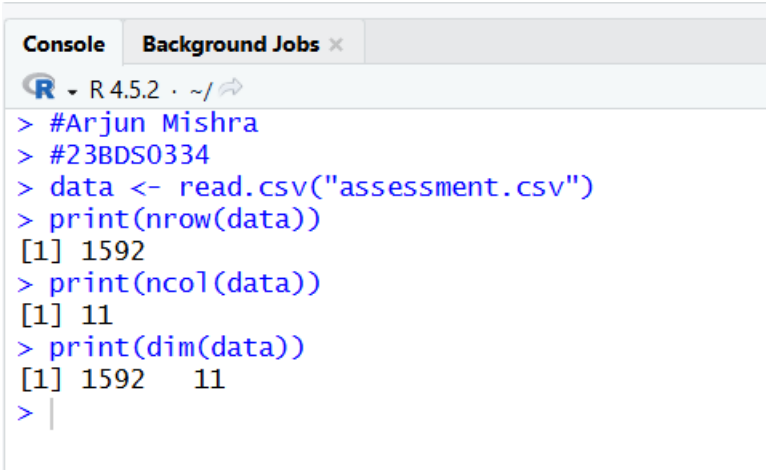
4. Find the total number of rows and columns.

AIM:
To find the total number of rows and columns in the dataset**.**

CODE:



OUTPUT:

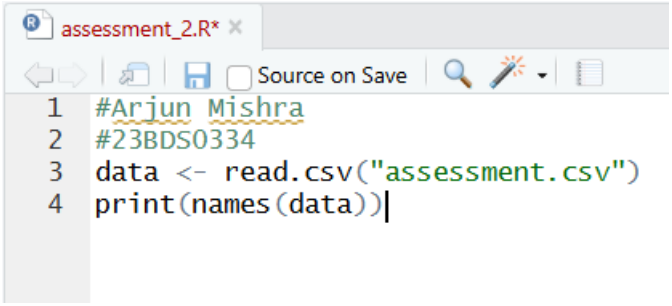


RESULT:
Thus, the total number of rows and columns was successfully determined.

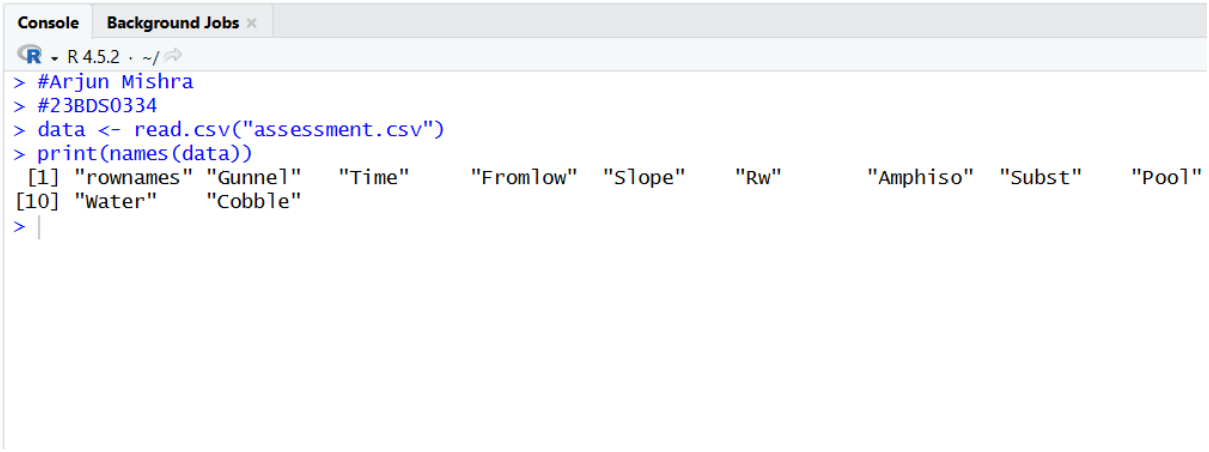
5. **Display the names of all variables (columns).**

AIM:
To display the names of all variables in the dataset.

CODE:



OUTPUT:



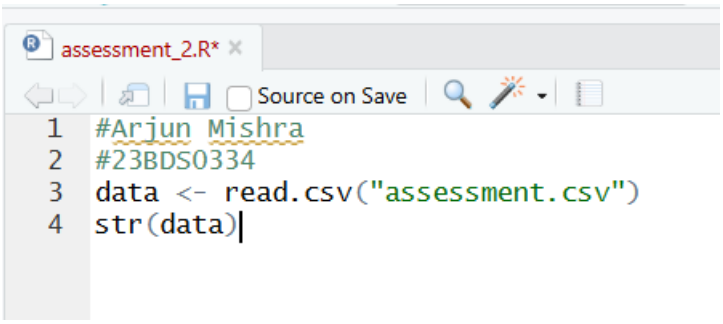
RESULT:
Thus, the variable names were successfully displayed.

6. **Display the data type (structure) of each variable.**

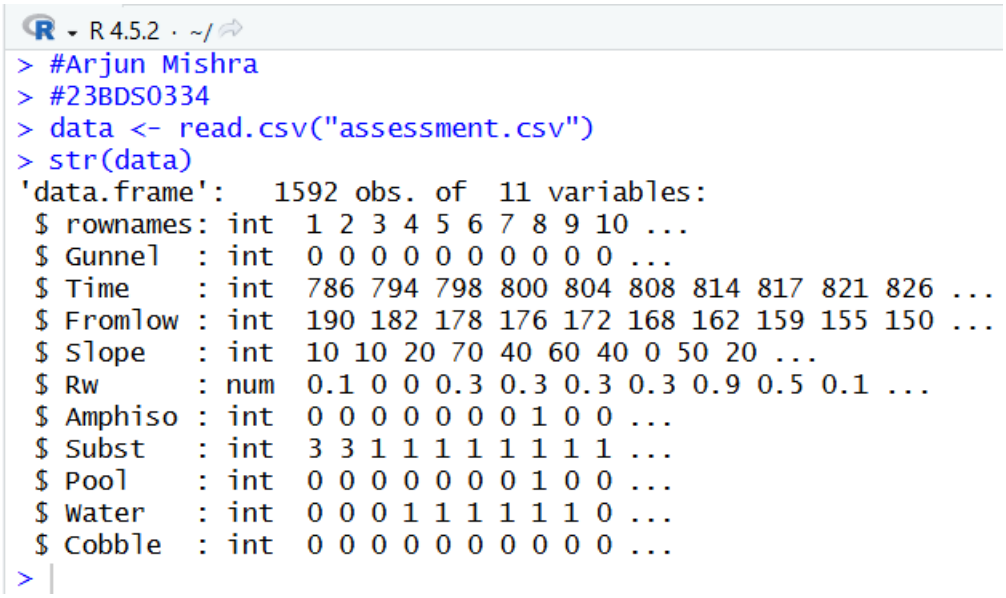
AIM:

To display the data type and structure of each variable.

CODE:



OUTPUT:



RESULT:

Thus, the structure and data types of all variables were successfully identified.

7. Display the summary statistics of the dataset.

AIM:

To display the summary statistics of the dataset.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
print(summary(data))
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> print(summary(data))
    rownames          Gunnel             Time          Fromlow          Slope             Rw        
 Min.   :   1.0   Min.   :0.00000   Min.   :340.0   Min.   :  0.00   Min.   : 0.000   Min.   :0.0000  
 1st Qu.: 398.8   1st Qu.:0.00000   1st Qu.:623.0   1st Qu.: 71.75   1st Qu.: 0.000   1st Qu.:0.0000  
 Median : 796.5   Median :0.00000   Median :712.5   Median :158.00   Median : 0.000   Median :0.6000  
 Mean   : 796.5   Mean   :0.03015   Mean   :705.2   Mean   :149.41   Mean   : 9.504   Mean   :0.5271  
 3rd Qu.:1194.2   3rd Qu.:0.00000   3rd Qu.:810.0   3rd Qu.:224.25   3rd Qu.:10.000   3rd Qu.:1.0000  
 Max.   :1592.0   Max.   :1.00000   Max.   :983.0   Max.   :304.00   Max.   :90.000   Max.   :1.0000  
    Amphiso           Subst             Pool             Water            Cobble      
 Min.   :0.0000   Min.   : 1.000   Min.   :0.0000   Min.   :0.0000   Min.   :0.000  
 1st Qu.:0.0000   1st Qu.: 1.000   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.000  
 Median :0.0000   Median : 2.000   Median :0.0000   Median :1.0000   Median :0.000  
 Mean   :0.4642   Mean   : 3.106   Mean   :0.2023   Mean   :0.5088   Mean   :0.191  
 3rd Qu.:1.0000   3rd Qu.: 4.000   3rd Qu.:0.0000   3rd Qu.:1.0000   3rd Qu.:0.000  
 Max.   :4.0000   Max.   :13.000   Max.   :1.0000   Max.   :1.0000   Max.   :1.000  
> 
```

RESULT:

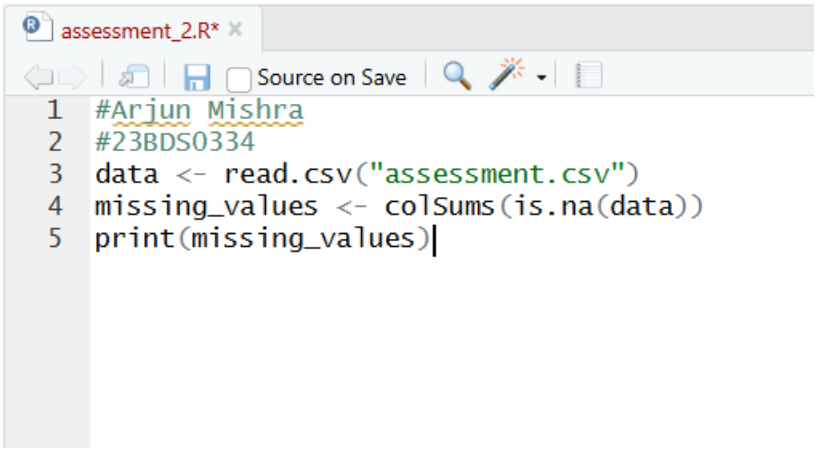
Thus, summary statistics of the dataset were successfully obtained.

8. Determine the number of missing values in each column.

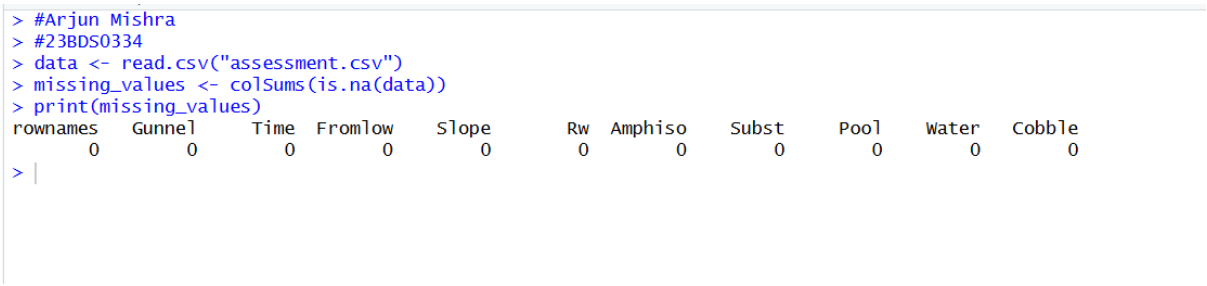
AIM:

To determine the number of missing values in each column.

CODE:



OUTPUT:



RESULT:

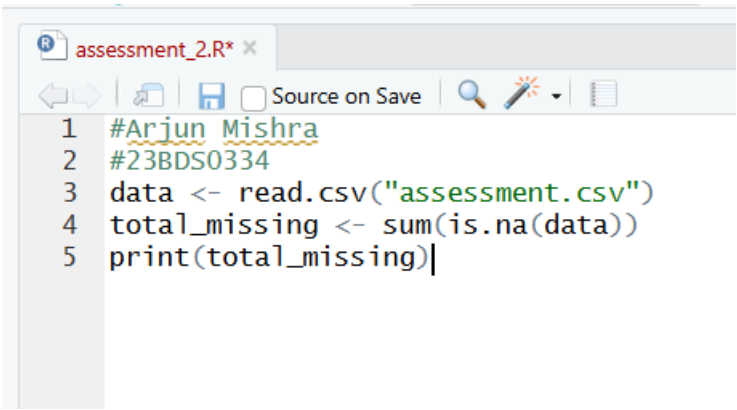
Thus, the missing values in each column were successfully determined.

9. Display the total number of missing values in the dataset.

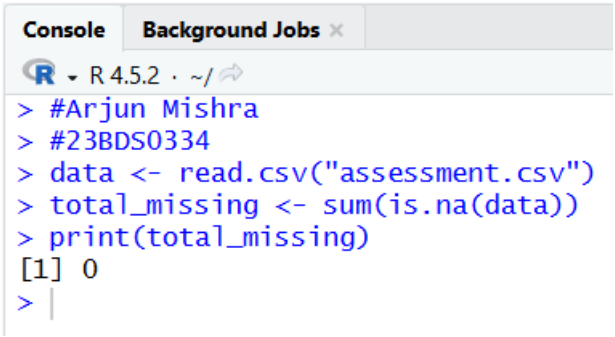
AIM:

To determine the total number of missing values in the entire dataset.

CODE:



OUTPUT:



RESULT:

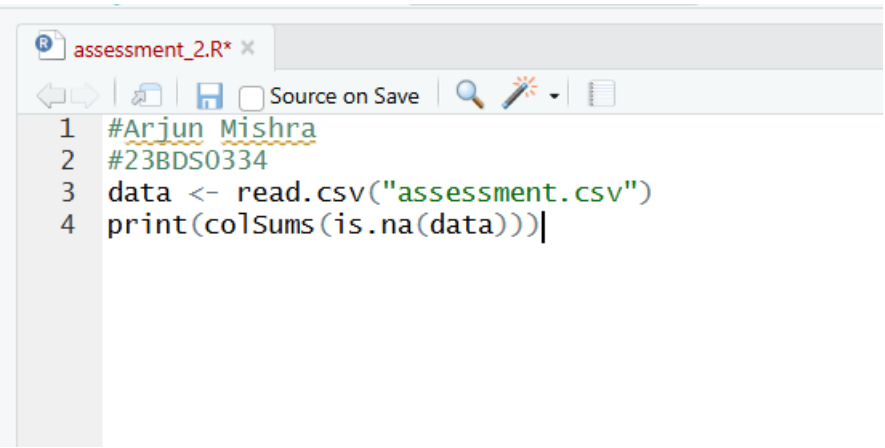
Thus, the total number of missing values was successfully calculated.

10. Display the number of missing values in each column.

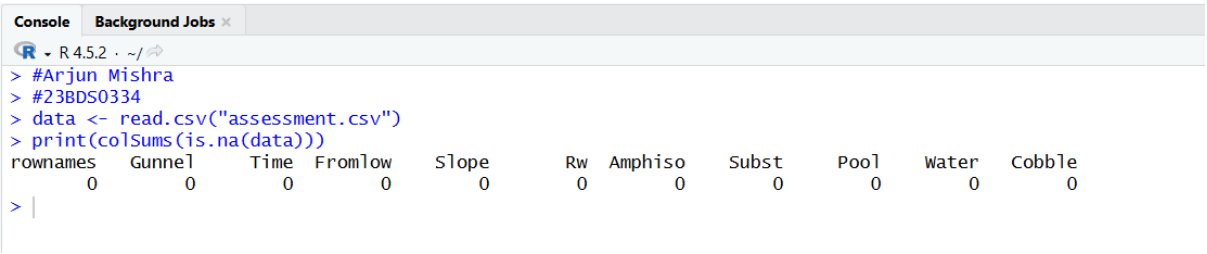
AIM:

To display the number of missing values present in each column.

CODE:



OUTPUT:



RESULT:

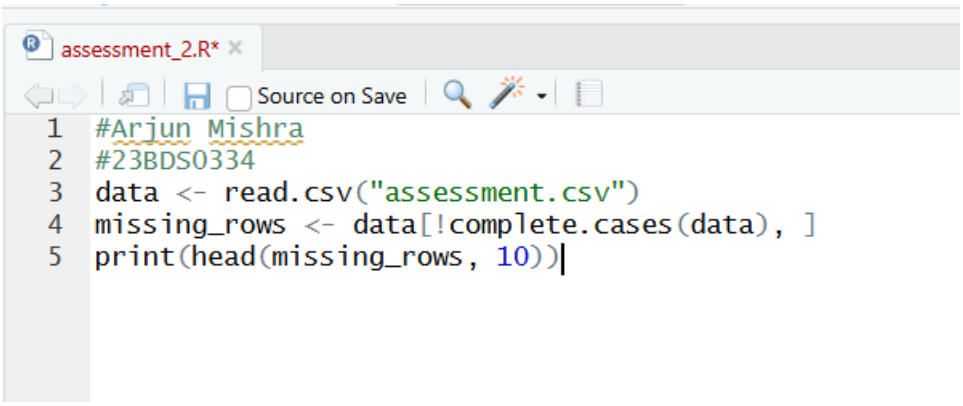
Thus, column-wise missing values were successfully displayed.

11. Identify the rows containing at least one missing value.

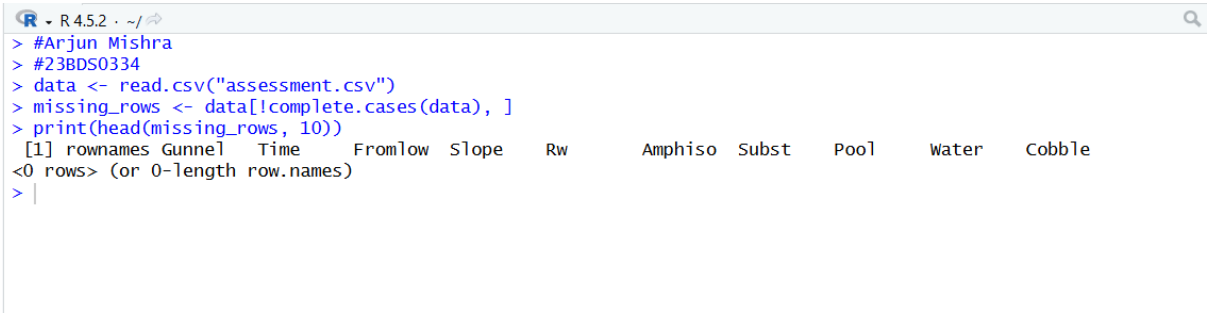
AIM:

To identify rows containing at least one missing value.

CODE:



OUTPUT:



RESULT:

Thus, rows containing missing values were successfully identified.

12. Remove all rows containing missing values and display the cleaned dataset.

AIM:

To remove all rows containing missing values and obtain a cleaned dataset.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
clean_data <- na.omit(data)
print(head(clean_data, 10))
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> clean_data <- na.omit(data)
> print(head(clean_data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, all rows containing missing values were successfully removed.

13. Replace missing values in a selected numerical column using the mean of that column.

AIM:

To replace missing values in a selected numerical column with its mean.

Code:

CODE:

assessment_2.R*

Source on Save

```r
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

mean_value <- mean(data$Time, na.rm = TRUE)

data$Time[is.na(data$Time)] <- mean_value

print(head(data$Time, 10))
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> mean_value <- mean(data$Time, na.rm = TRUE)
>
> data$Time[is.na(data$Time)] <- mean_value
>
> print(head(data$Time, 10))
 [1] 786 794 798 800 804 808 814 817 821 826
>
```

RESULT:

Thus, missing values in the selected numerical column were successfully replaced using the mean.

14. Replace missing values in another numerical column using the median.

AIM:

To replace missing values in another numerical column using its median.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
median_value <- median(data$Slope, na.rm = TRUE)
data$Slope[is.na(data$Slope)] <- median_value
print(head(data$Slope, 10))
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> median_value <- median(data$Slope, na.rm = TRUE)
> data$Slope[is.na(data$Slope)] <- median_value
> print(head(data$Slope, 10))
 [1] 10 10 20 70 40 60 40  0 50 20
>
```

RESULT:

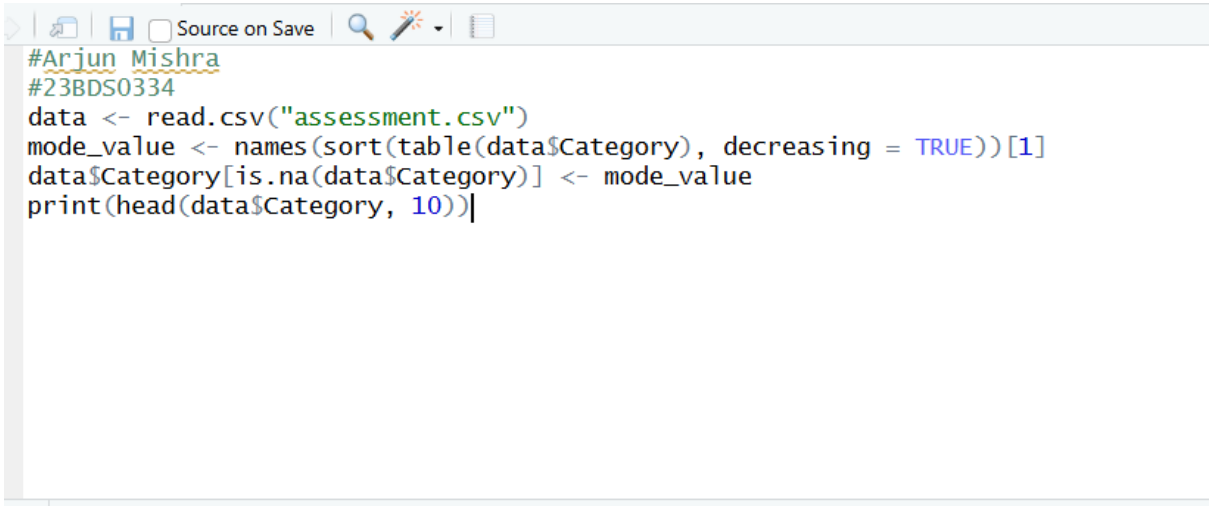
Thus, missing values were successfully replaced using the median.

15. Replace missing values in a categorical column using the mode.

AIM:

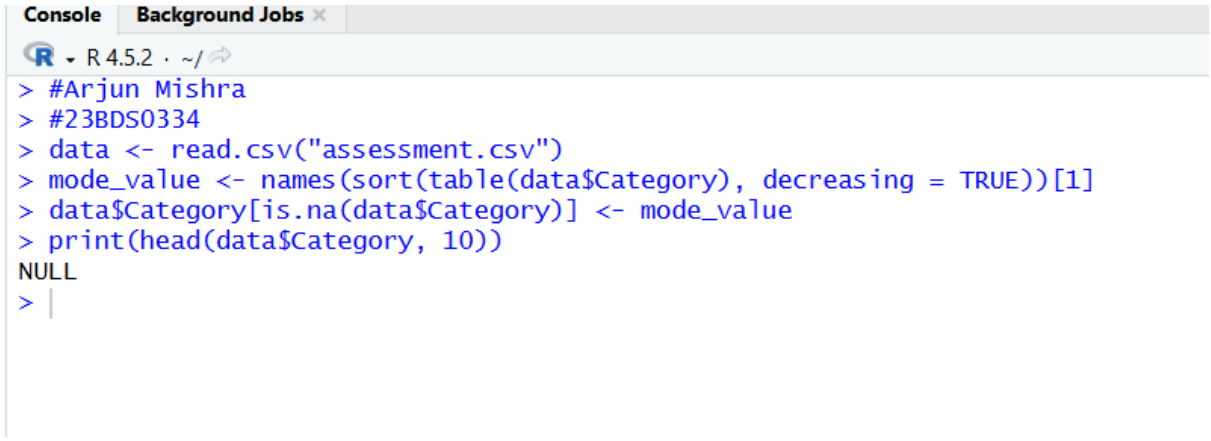
To replace missing values in a categorical column using the mode.

CODE:



```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
mode_value <- names(sort(table(data$Category), decreasing = TRUE))[1]
data$Category[is.na(data$Category)] <- mode_value
print(head(data$Category, 10))
```

OUTPUT:



```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> mode_value <- names(sort(table(data$Category), decreasing = TRUE))[1]
> data$Category[is.na(data$Category)] <- mode_value
> print(head(data$Category, 10))
NULL
>
```

RESULT:

Thus, missing categorical values were successfully replaced using the mode.

16. Perform forward filling to replace missing values with the previous non-missing value. (Load the package zoo, and use the function locf())

AIM:

To replace missing values with the previous non-missing value using forward filling.

CODE:

assessment_2.R*

Source on Save

```r
#Arjun Mishra
#23BDS0334
install.packages("zoo")
library(zoo)

data <- read.csv("assessment.csv")
data$Time <- na.locf(data$Time, na.rm = FALSE)
print(head(data$Time, 10))
```

OUTPUT:

```
https://cran.r-project.org/doc/manuals/r-patched/R-admin.html#Inst
> library(zoo)

Attaching package: 'zoo'

The following objects are masked from 'package:base':

    as.Date, as.Date.numeric

Warning message:
package 'zoo' was built under R version 4.5.3
>
> data <- read.csv("assessment.csv")
> data$Time <- na.locf(data$Time, na.rm = FALSE)
> print(head(data$Time, 10))
 [1] 786 794 798 800 804 808 814 817 821 826
>
```

RESULT:

Thus, forward filling was successfully performed using na.locf().

17. Perform backward filling to replace missing values with the next non-missing value.

AIM:

To replace missing values with the next available non-missing value.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
library(zoo)

data <- read.csv("assessment.csv")
data$Time <- na.locf(data$Time, fromLast = TRUE, na.rm = FALSE)
print(head(data$Time, 10))
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> library(zoo)
>
> data <- read.csv("assessment.csv")
> data$Time <- na.locf(data$Time, fromLast = TRUE, na.rm = FALSE)
> print(head(data$Time, 10))
 [1] 786 794 798 800 804 808 814 817 821 826
>
```

RESULT:

Thus, backward filling was successfully performed.

18. Replace remaining missing values with a constant value (0 or "Unknown").

AIM:

To replace remaining missing numerical values with 0 and character values with "Unknown".

CODE:

assessment_2.R*

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

for (i in names(data)) {
  if (is.numeric(data[[i]])) {
    data[[i]][is.na(data[[i]])] <- 0
  } else {
    data[[i]][is.na(data[[i]])] <- "Unknown"
  }
}

print(head(data, 10))
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> for (i in names(data)) {
+   if (is.numeric(data[[i]])) {
+     data[[i]][is.na(data[[i]])] <- 0
+   } else {
+     data[[i]][is.na(data[[i]])] <- "Unknown"
+   }
+ }
>
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, remaining missing values were successfully replaced with constant values.

19. Remove duplicate records from the dataset.

AIM:

To identify and remove duplicate records from the dataset.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")
print(sum(duplicated(data)))
data <- data[!duplicated(data), ]
print(head(data, 10))
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> print(sum(duplicated(data)))
[1] 0
> data <- data[!duplicated(data), ]
> print(head(data, 10))
   rownames Gunnel Time Fromlow Slope  Rw Amphiso Subst Pool Water Cobble
1         1      0  786     190    10 0.1       0     3    0     0      0
2         2      0  794     182    10 0.0       0     3    0     0      0
3         3      0  798     178    20 0.0       0     1    0     0      0
4         4      0  800     176    70 0.3       0     1    0     1      0
5         5      0  804     172    40 0.3       0     1    0     1      0
6         6      0  808     168    60 0.3       0     1    0     1      0
7         7      0  814     162    40 0.3       0     1    0     1      0
8         8      0  817     159     0 0.9       1     1    1     1      0
9         9      0  821     155    50 0.5       0     1    0     1      0
10       10      0  826     150    20 0.1       0     1    0     0      0
>
```

RESULT:

Thus, duplicate records were successfully removed.

20. Convert appropriate variables into factors.

AIM:

To convert an appropriate categorical variable into a factor.

CODE:

assessment_2.R*

Source on Save

```
#Arjun Mishra
#23BDS0334
data <- read.csv("assessment.csv")

char_columns <- sapply(data, is.character)

data[char_columns] <- lapply(data[char_columns], as.factor)

str(data)
```

OUTPUT:

Console | Background Jobs

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
>
> char_columns <- sapply(data, is.character)
>
> data[char_columns] <- lapply(data[char_columns], as.factor)
>
> str(data)
'data.frame':   1592 obs. of  11 variables:
 $ rownames: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Gunnel  : int  0 0 0 0 0 0 0 0 0 0 ...
 $ Time    : int  786 794 798 800 804 808 814 817 821 826 ...
 $ Fromlow : int  190 182 178 176 172 168 162 159 155 150 ...
 $ Slope   : int  10 10 20 70 40 60 40 0 50 20 ...
 $ Rw      : num  0.1 0 0 0.3 0.3 0.3 0.3 0.9 0.5 0.1 ...
 $ Amphiso : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Subst   : int  3 3 1 1 1 1 1 1 1 1 ...
 $ Pool    : int  0 0 0 0 0 0 0 1 0 0 ...
 $ Water   : int  0 0 0 1 1 1 1 1 1 0 ...
 $ Cobble  : int  0 0 0 0 0 0 0 0 0 0 ...
>
```

RESULT:

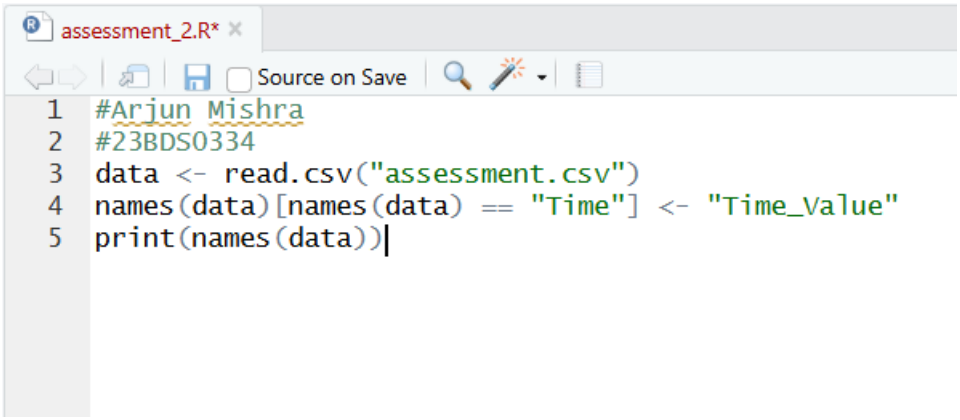
Thus, the selected categorical variable was successfully converted into a factor.

21. Rename one or more column names.

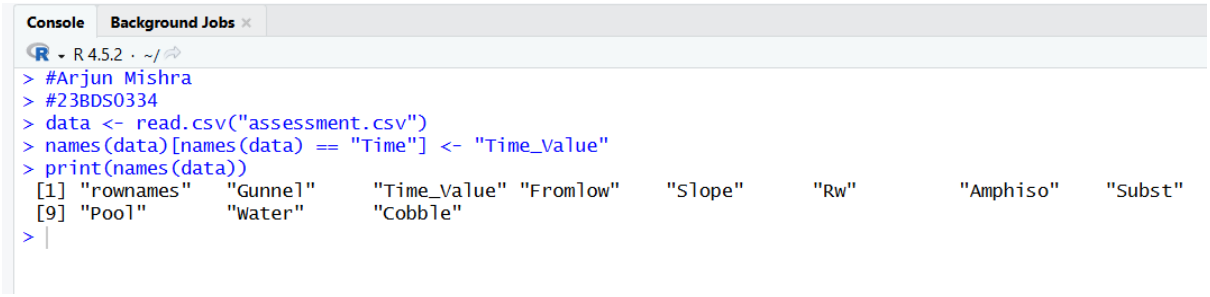
AIM:

To rename one or more columns of the dataset.

CODE:



OUTPUT:



RESULT:

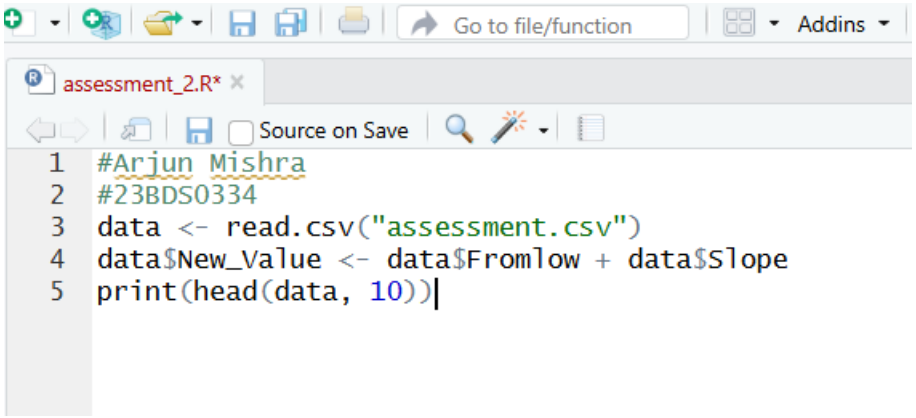
Thus, the selected column was successfully renamed.

22. Create a new derived column using existing variables.

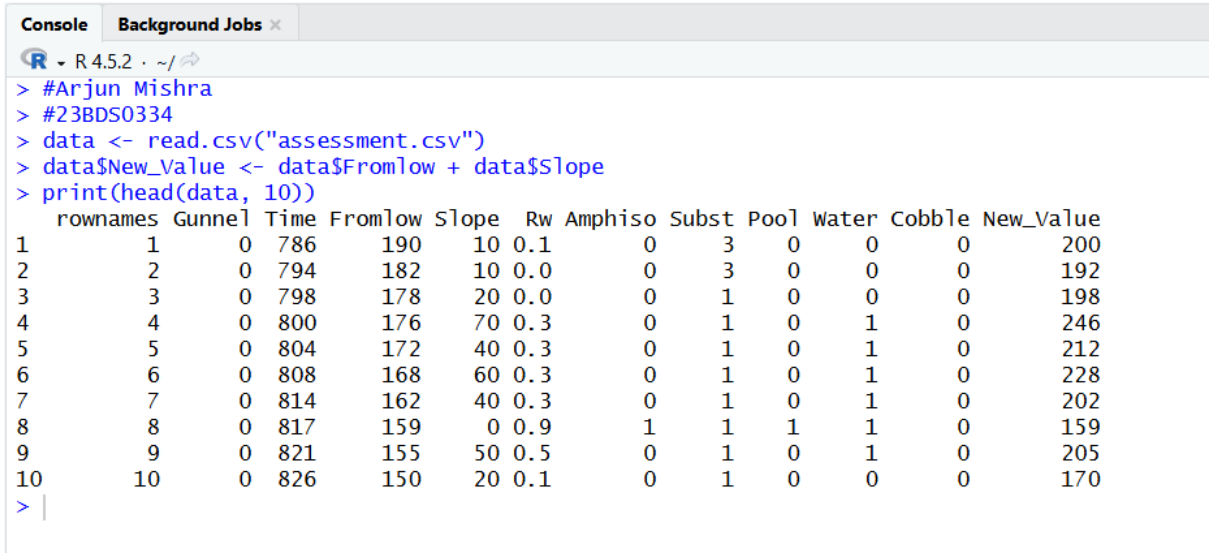
AIM:

To create a new derived column using existing variables.

CODE:



OUTPUT:



RESULT:

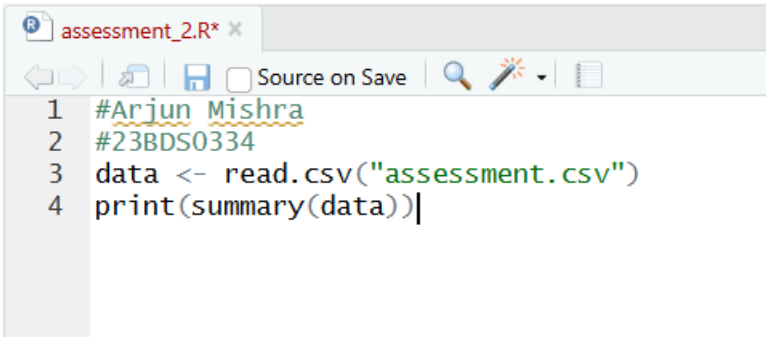
Thus, a new derived variable was successfully created using existing numerical variables.

23. Display the summary statistics of the dataset using summary().

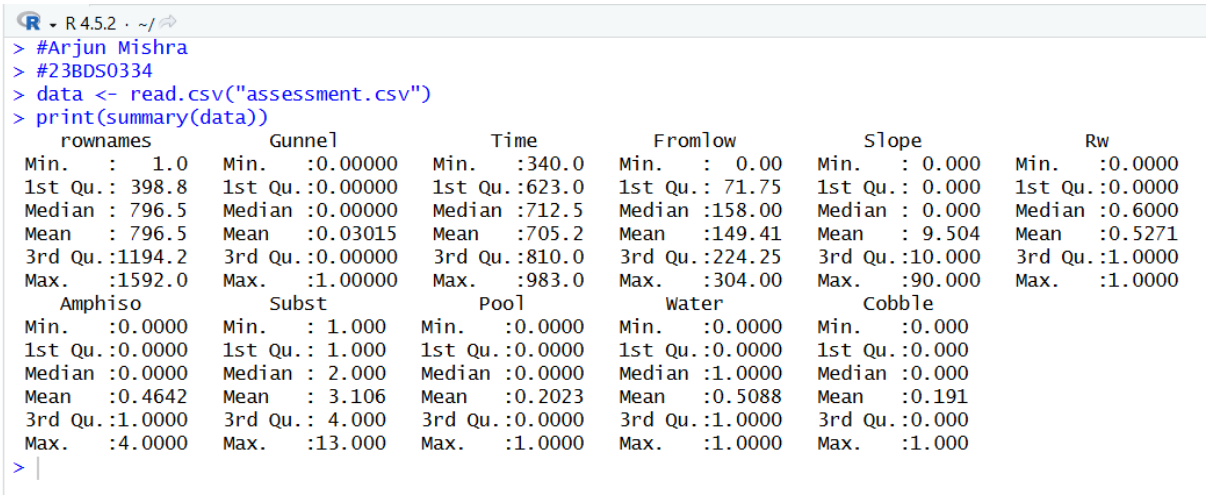
AIM:

To display the summary statistics of the dataset using summary().

CODE:



OUTPUT:



RESULT:

Thus, the summary statistics were successfully calculated using summary().

24. Find the minimum and maximum values.

AIM:

To find the minimum and maximum values of a numerical variable.

CODE:

assessment_2.R*

Source on Save

```
1  #Arjun Mishra
2  #23BDS0334
3  data <- read.csv("assessment.csv")
4  minimum <- min(data$Time, na.rm = TRUE)
5  maximum <- max(data$Time, na.rm = TRUE)
6  print(minimum)
7  print(maximum)
```

OUTPUT:

R 4.5.2 · ~/

```
> #Arjun Mishra
> #23BDS0334
> data <- read.csv("assessment.csv")
> minimum <- min(data$Time, na.rm = TRUE)
> maximum <- max(data$Time, na.rm = TRUE)
> print(minimum)
[1] 340
> print(maximum)
[1] 983
>
```

RESULT:

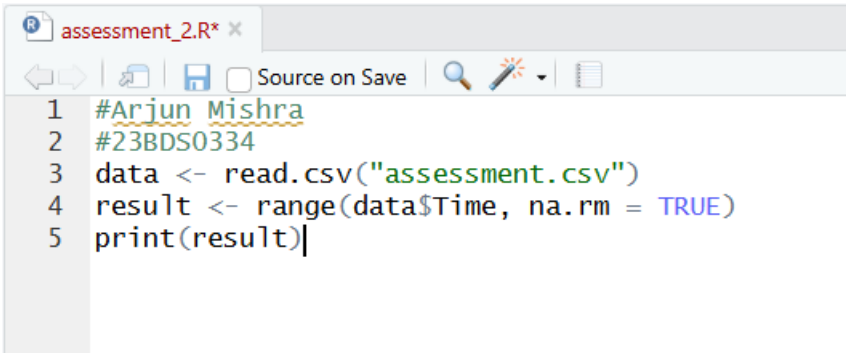
Thus, the minimum and maximum values were successfully calculated.

25. Calculate the range of a numerical variable.

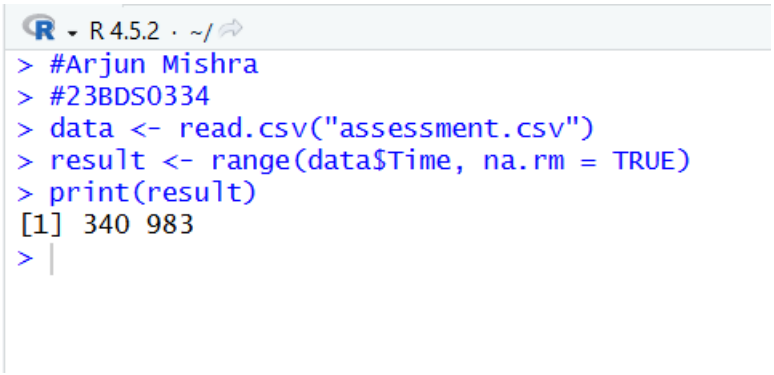
AIM:

To calculate the range of a numerical variable.

CODE:



OUTPUT:



RESULT:

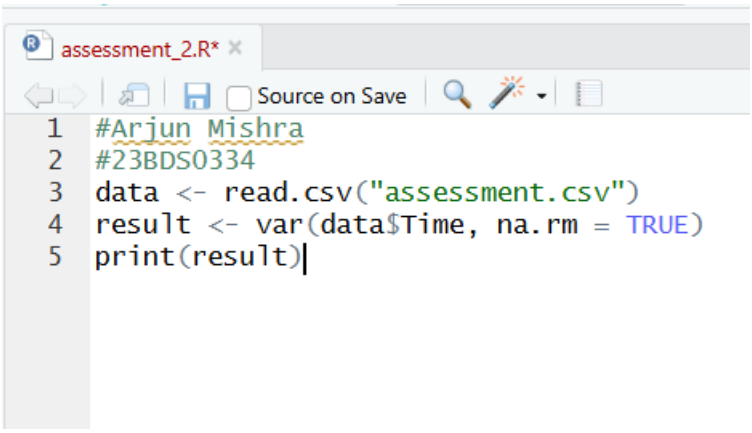
Thus, the range of the selected numerical variable was successfully calculated.

26. Calculate the variance.

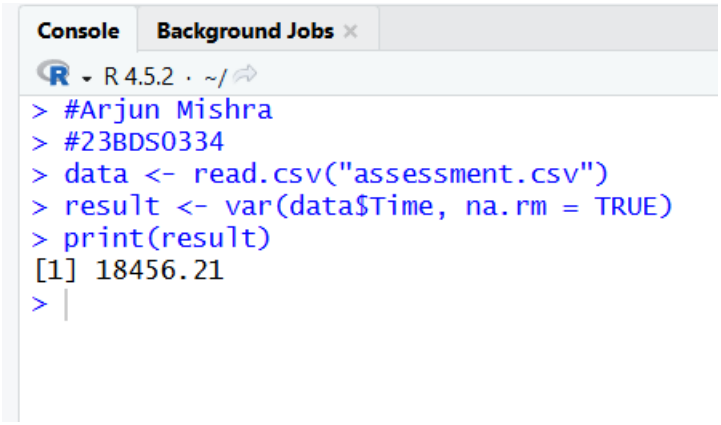
AIM:

To calculate the variance of a numerical variable.

CODE:



OUTPUT:



RESULT:

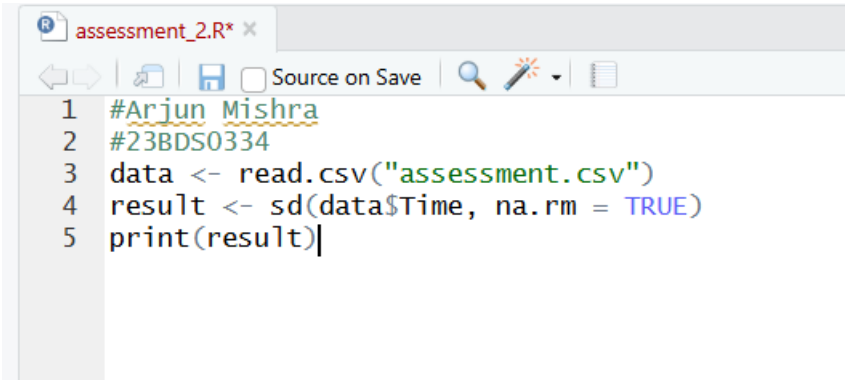
Thus, the variance was successfully calculated.

27. Calculate the standard deviation.

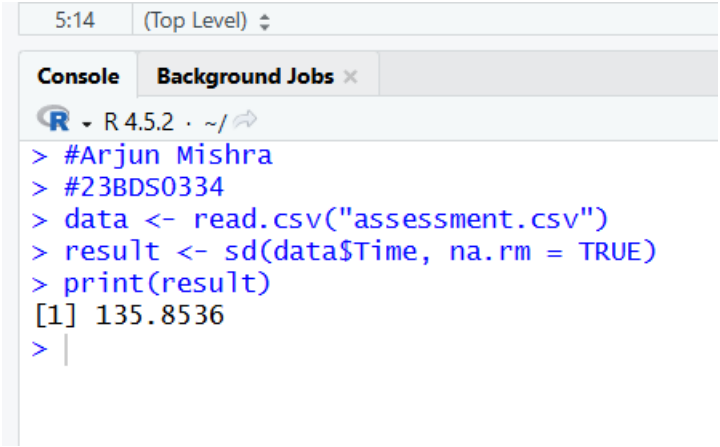
AIM:

To calculate the variance of a numerical variable.

CODE:



OUTPUT:



RESULT:

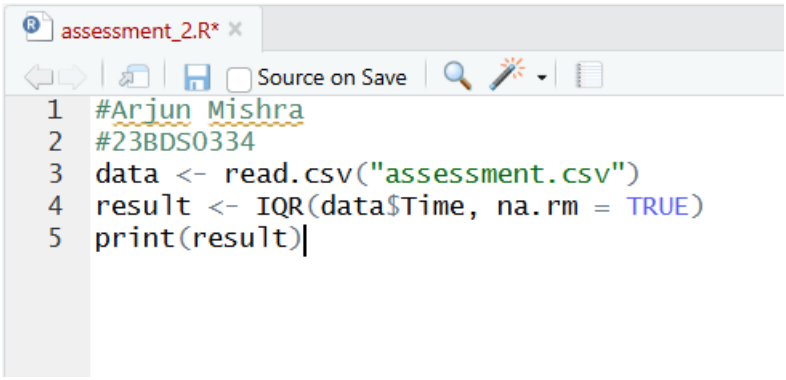
Thus, the standard deviation was successfully calculated.

28. Calculate the interquartile range (IQR).

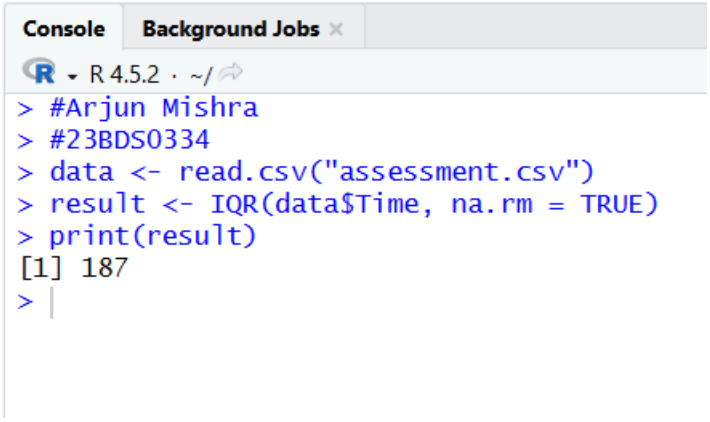
AIM:

To calculate the interquartile range of a numerical variable.

CODE:



OUTPUT:



RESULT:

Thus, the interquartile range was successfully calculated using IQR().